

AWS Deployment Architecture

To deploy this MERN stack Mini Expense Tracker efficiently, I use a managed-services approach.

Frontend screen:

Mini Expense Tracker

Add Expense

food

▼

Add Expense

Summary

food: \$241.00

travel: \$45.00

shopping: \$65.99

other: \$30.00

bills: \$120.75

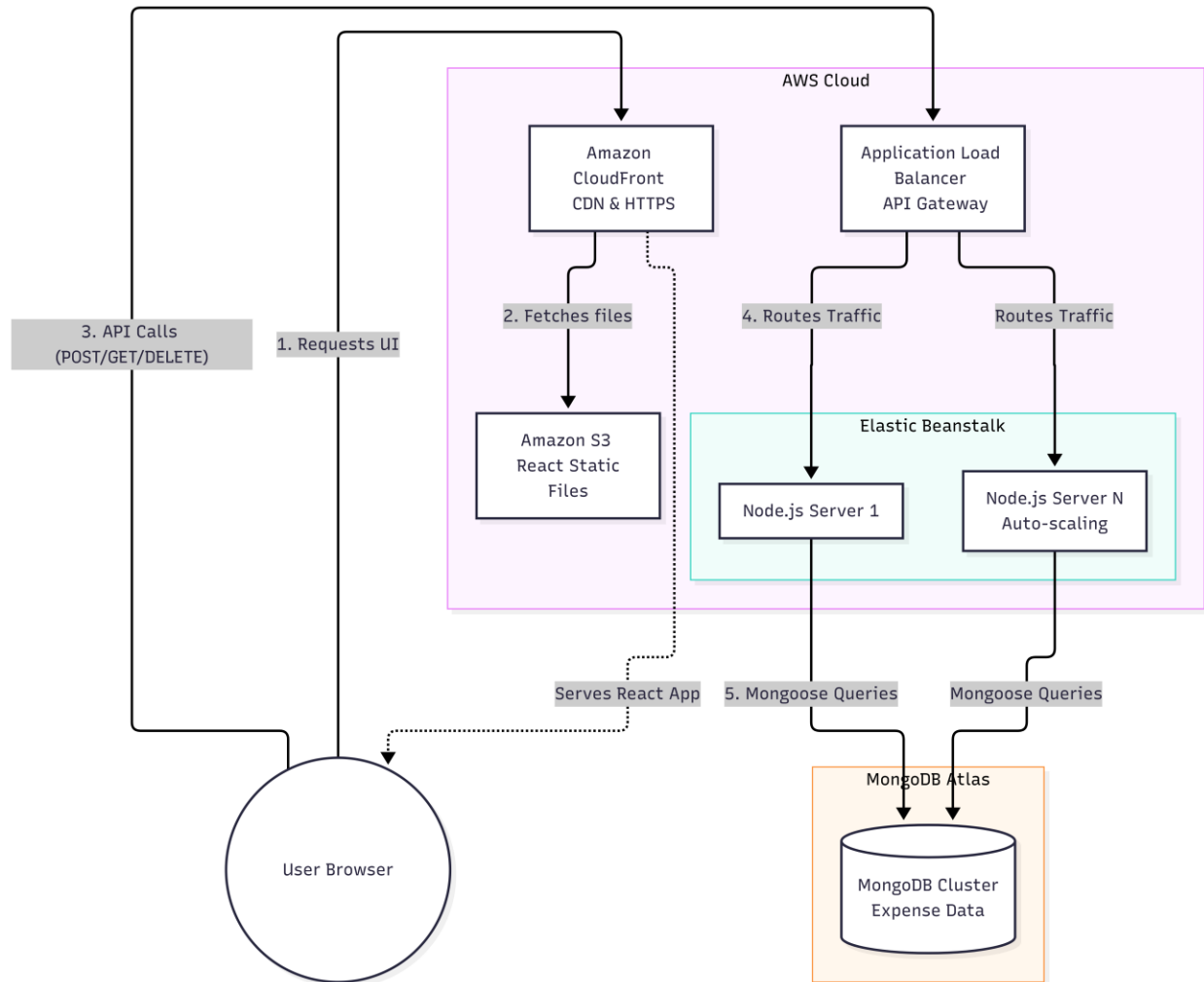
Grand Total: \$502.74

Expenses

All Categories ▼

Title	Amount	Category	Date	Action
Gym Membership	\$30.00	other	9/19/2026	Delete
New Shoes	\$65.99	shopping	9/19/2026	Delete
Electric Bill	\$120.75	bills	9/19/2026	Delete
Uber to Airport	\$45.00	travel	9/19/2026	Delete
Pizza	\$220.50	food	9/19/2026	Delete
Pizza	\$20.50	food	9/19/2026	Delete

System Architecture: I attached mermaid code that i built on **mermaid**



- **Frontend (React App): Amazon S3 + CloudFront**
 - Build React app into static HTML/CSS/JS files and store them in an **Amazon S3** bucket. As S3 service do not require manage it by hand.
 - Place **Amazon CloudFront** (a Content Delivery Network) in front of S3. It caches frontend on servers globally so the UI loads instantly, and it provides a free SSL certificate for secure HTTPS connections.
- **Backend (Node.js/Express): AWS Elastic Beanstalk**
 - Deploy your server .js to **Elastic Beanstalk**. This is a Platform-as-a-Service (PaaS) that automatically sets up the necessary EC2 virtual servers to run your code.

- It automatically provisions an **Application Load Balancer (ALB)** to manage incoming API requests and distribute them across your servers.
- **Database (MongoDB): MongoDB Atlas**
 - Use **MongoDB Atlas** and select the exact same AWS Region as your Elastic Beanstalk servers (e.g., us-east-1). Keeping them in the same region ensures near-zero latency when your backend talks to your database.

II. Overall Working Data Flow

1. **UI Delivery:** The user navigates to your domain. CloudFront intercepts the request and instantly serves the cached React application from the S3 bucket.
2. **User Action:** The user fills out the "Add Expense" form and clicks submit.
3. **API Request:** React makes an HTTP POST request to your backend domain.
4. **Load Balancing:** The AWS Application Load Balancer receives the request and forwards it to a healthy Node.js server.
5. **Database Execution:** Express validates the request body using Mongoose. It opens a connection to MongoDB Atlas, saves the document, and returns a 201 status code.
6. **UI Reaction:** The React app receives the success response, clears the form, and triggers parallel GET requests to instantly update the list and the summary aggregation.